深度研究报告

video-note 字幕获取方案调研

围绕 yt-dlp、YouTube 专用库与 B 站专用方案的维护者决策建议

Codex

2026-03-26

本报告不使用封面图，重点保留方法、证据与可执行结论。

核心问题：现有 pipeline 是否应该继续以 yt-dlp 作为字幕获取核心，还是应替换为更专用的库或 B 站专用实现？
研究范围：关注 YouTube 与 Bilibili 两个平台；目标是为 `video-note-pipeline` 的下一轮 change 提供可执行建议，而不是做泛化爬虫综述。
目标读者：`video-note-pipeline` 与 `video-note-render-pdf` 的维护者
证据窗口：外部资料访问时间为 2026-03-26；本地实现基线为 `video-note-pipeline` commit `75ab50e574c6bf63e3ad1baf0486f3ed3b057461`

目录

1 执行摘要 **2**

2 研究范围与方法 **2**

3 当前实现的真正问题在哪里 **3**

3.1 证据一：当前 pipeline 在 cookies 有效时仍判定无字幕 3

3.2 证据二：直接调用 yt-dlp 能拿到 `ai-zh` 字幕 4

4 外部方案比较 **4**

4.1 `yt-dlp`：跨平台后备层，而不是 transcript-first 层 4

4.2 `youtube-transcript-api`：YouTube 上更像正确的第一层 5

4.3 `bilibili-api-python`：Python 原生，但属于更重的 B 站专用层 5

4.4 `BBDown`：B 站专用 CLI，适合作为外部 fallback，而不是马上内嵌 6

4.5 比较表 6

5 建议的目标架构 **7**

5.1 YouTube：维持现状的主方向，不建议大改 7

5.2 Bilibili：先修策略，再决定是否引入专用层 7

5.3 什么时候再考虑 B 站专用库 8

6 为后续 propose 准备的变更建议 **8**

7 结论与后续问题 **8**

7.1 已经有强证据支持的判断 8

7.2 很可能成立，但还值得继续验证的判断 8

7.3 还不知道，但下一轮 change 应该顺手补上的观测点 9

A 来源清单 **9**

1 执行摘要

结论先行

不建议现在把 `yt-dlp` 整体替换掉。对于 B 站，当前漏字幕的主因是本地 adapter 的语言请求与归一化策略过窄，而不是 `yt-dlp` 本身拿不到字幕。对于 YouTube，`youtube-transcript-api` 依然有明显优势，应保留在第一优先级。

- 强支持结论：`yt-dlp` 官方支持列出字幕、按正则或 `all` 请求语言、自动字幕、浏览器 cookies 导入（S1）。这意味着它具备作为跨平台字幕后备层的基础能力。
- 强支持结论：`youtube-transcript-api` 官方 README 明确支持列出 transcript、区分人工/自动字幕、翻译字幕，且不需要 headless browser（S2）。它适合作为 YouTube 的 transcript-first 路径。
- 强支持结论：本地复现实验显示，当前 pipeline 在 cookies 有效的情况下，对样例 `BV1DzALzQEHU` 仍返回 `selected_track.source = none`(L2)；但直接调用 `yt-dlp` 并使用 `--sub-langs all,-live_chat,-danma` 可以成功拿到 `ai-zh.srt`（L3）。这直接指向“请求/筛选策略错误”而不是“cookies 或 yt-dlp 不可用”。
- 审慎结论：B 站专用方案如 `bilibili-api-python`（S4）和 `BBDown`（S3）都值得关注，但它们更适合作为第二阶段增强或专用 fallback，而不是现在就替换主干。

不该做的事

现在最不划算的选择，是为了修一个 `ai-zh` 漏检问题，立刻把整条 B 站字幕链替换成新的第三方库或外部 CLI。这样会引入更高的维护成本、部署复杂度和平台差异，却不能保证比修正现有策略更稳。

2 研究范围与方法

研究问题与判据

本报告不是在问“哪个工具功能最多”，而是在问“对于现有工程，哪个方案的维护收益最高”。判据有四个：

- 能否稳定拿到平台原生字幕
- 与现有 Python/CLI 架构的集成代价
- cookies、登录态与风控处理的复杂度
- 当平台变化时的维护成本

本次研究结合了三类证据：

- 本地实现证据：`video-note-pipeline` 当前 adapter、测试与真实 probe artifact
- 官方或项目一手资料：各库或工具自己的 README / 文档
- 针对 B 站 AI 字幕的公开社区证据：确认字幕标签与接口形态的存在

本地复现实验

研究使用的关键本地路径如下：

- 当前实现：`/home/fanghaotian/Desktop/src/video-note-pipeline/src/video_note_pipeline/adapters/base.py`
- 失败 probe artifact：`/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-subtitle-strategy/artifacts/current-subtitle-probe-BV1DzALzQEHU.json`
- 直接调用 `yt-dlp` 成功拿到的字幕：`/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-subtitle-strategy/artifacts/direct-yt-dlp-BV1DzALzQEHU.ai-zh.srt`

3 当前实现的真正问题在哪里

当前 B 站字幕下载路径的核心逻辑是：

- 通过 `yt-dlp` 下载 sidecar 字幕
- 只请求固定语言集合 `zh-Hans`，`zh`，`zh-CN`，`zh-TW`，`en`，`en-US`，`ja`
- 从 `requested_subtitles` 里按固定优先级选轨

这段策略在“平台语言码命名稳定”的情况下能工作，但一旦平台返回 `ai-zh`、`zh-Hant`、`yue`、`auto-zh` 之类未列入白名单的标签，就会出现“明明有字幕，但 probe 结果是 none”的假阴性。

```
DEFAULT_LANGUAGE_PRIORITY = [
    "zh-Hans", "zh", "zh-CN", "zh-TW", "en", "en-US", "ja"
]
```

Listing 1: 当前实现中的固定语言优先级，摘自本地 adapter

下面两份本地证据把问题钉死了。

3.1 证据一：当前 pipeline 在 cookies 有效时仍判定无字幕


```
"cookies": {
  "ok": false,
  "tracks": [],
  "error": "No subtitle track could be parsed"
},
"selected_track": {
  "source": "none",
  "language": "",
  "kind": "",
  "reason": "No subtitle track could be parsed | No subtitle track could be parsed"
},
"cookie_status": {
  "status": "valid",
```

```
  "valid": true,
  "critical_cookie_names_present": ["DedeUserID", "SESSDATA", "bili_jct"]
}
```

Listing 2: 本地失败 probe 结果的关键字段，来源 L2

这说明 cookies 不是根因。登录态已经有效，但轨道仍未命中。

3.2 证据二：直接调用 yt-dlp 能拿到 ai-zh 字幕

```
uv run python -m yt_dlp \
  --cookies /home/fanghaotian/.config/video-note-pipeline/cookies/bilibili.txt \
  --skip-download \
  --write-subs \
  --write-auto-subs \
  --sub-langs 'all,-live_chat,-danmaku' \
  --sub-format 'json3/srt/vtt/best' \
  -o '/tmp/bili-sub-check-BV1DzALzQEHU/%(id)s.%(ext)s' \
  'https://www.bilibili.com/video/BV1DzALzQEHU/?spm_id_from=333.1007.tianma.6-1-19.
      click&vd_source=8eeed36c8368d0ab0bb576c12209518e'
```

Listing 3: 本地直连命令，成功拿到 B 站样例视频的字幕

```
1
00:00:12,180 --> 00:00:18,180
♪ 火 等 的 模 样 好 不 具 想 ♪

2
00:00:20,520 --> 00:00:26,800
♪ 你 用 皮 肤 感 受 你 的 流 向 ♪
```

Listing 4: 成功下载到的字幕文件片段，来源 L3

直接推论

对当前样例而言，`yt-dlp` 不是瓶颈。真正该修的是 B 站 adapter 的“请求哪些语言”和“如何把 `ai-zh` 归类为中文自动字幕候选”。

4 外部方案比较

4.1 yt-dlp：跨平台后备层，而不是 transcript-first 层

根据 `yt-dlp` 官方 README：

- 它支持 `--write-subs`、`--write-auto-subs`、`--list-subs`（S1）。
- 它支持 `--sub-langs`，并且语言参数可以是正则或 `all`；官方示例直接给出了 `all,-live_chat`（S1）。
- 它支持 `--cookies-from-browser` 并列出了 Linux Chromium keyring 的选项（S1）。

这三个点对当前工程非常关键。它们意味着：

- 我们不必自己实现浏览器 cookies 解密
- 我们不必自己维护所有平台的下载/字幕 sidecar 细节
- 我们有条件先拿全量字幕标签，再在本地做平台特定的归一化与排序

`yt-dlp` 的边界

`yt-dlp` 更像“通用提取器与下载器”，不是“语义化 transcript SDK”。不同 extractor 暴露的语言码和 sidecar 命名规则并不统一，所以如果上层策略偷懒，把语言表写死，误判就会落到我们自己头上。

4.2 `youtube-transcript-api`：YouTube 上更像正确的第一层

根据 `youtube-transcript-api` 官方 README：

- 它直接面向 transcript/subtitle 语义，支持人工字幕、自动字幕、翻译字幕（S2）。
- 它提供 `list(video_id)`、`find_transcript()`、`find_manually_created_transcript()`、`find_generated_tra` 这类高层接口（S2）。
- 它不需要 API key，也不需要 headless browser（S2）。

这就是它比 `yt-dlp` 更适合 YouTube 第一层的原因：不是功能覆盖更广，而是返回的数据结构更接近我们真正需要的 transcript 模型。

但 README 也明确提醒了两个限制：

- 它使用的是 undocumented YouTube web-client API，因此没有长期稳定性保证（S2）。
- 当前 cookie-based authentication 因 YouTube 的近期变更而不可用（S2）。

对 YouTube 的维护判断

YouTube 平台不需要“只保留一个工具”。最佳工程策略仍然是：`youtube-transcript-api` 优先，`yt-dlp` 作 cookie/sidecar 补位，最后才进入 ASR。

4.3 `bilibili-api-python`：Python 原生，但属于更重的 B 站专用层

从 `bilibili-api-python` 项目 README 可见：

- 它明确把“下载字幕文件”列在能力范围内（S4）。
- 它也覆盖 cookies 刷新、代理、反风控规避、异步请求库选择等工程问题（S4）。
- README 自己也提醒，这类接口变更后可能会马上失效，需要持续更新（S4）。

这说明它适合什么，不适合什么：

- 适合：当我们确定 B 站需要更细粒度的 `cid`、字幕列表、登录态与反风控控制时，作为 Python 内部的 B 站专用增强层
- 不适合：为了修一个现成可复现的语言筛选问题，直接替换掉现有 `yt-dlp` 主干

维护代价

这类库的价值在“B 站语义更强”，不是“天然更稳”。它本质上仍然依赖非官方接口。只要 B 站网页或客户端接口变化，维护压力就会回到我们自己身上。

4.4 `BBDown`：B 站专用 CLI，适合作为外部 fallback，而不是马上内嵌

根据 `BBDown` README：

- 它支持 `--sub-only`，可只下载字幕（S3）。
- 它支持 AI 字幕，但默认开启 `--skip-ai`，需要显式调整策略（S3）。
- 它支持扫码登录、手工传 cookie、甚至 TV / App 鉴权路径（S3）。

这很有吸引力，因为它明显更懂 B 站。但它的问题也同样明显：

- 它是独立 CLI，不是我们现有 Python 依赖的一部分
- 引入它会增加二进制管理、跨平台安装、日志解析与故障诊断复杂度
- 它的“更懂 B 站”在短期内并不能证明比修正 `yt-dlp` 调用策略更划算

4.5 比较表

方案	主要定位	明显优势	主要代价 / 风险
`yt-dlp`	跨平台提取与下载	平台覆盖广；支持 cookies-from-browser；支持列字幕和请求全量语言；已在现有工程内使用	语言码与 sidecar 细节需上层自己归一化；不是 transcript-first 数据模型
`youtube-transcript-api`	YouTube transcript SDK	transcript 语义直接；可区分人工/自动字幕；不需要 headless browser	仅适用于 YouTube；使用 undocumented API；cookie 认证当前不可用
`bilibili-api-python`	Python 原生 B 站专用 API 层	可直接处理 B 站语义；覆盖字幕文件、Credential、代理、异步请求	非官方接口；维护压力更高；引入复杂度高于修正现有 adapter
`BBDown`	B 站专用外部 CLI	对 B 站下载、登录、字幕场景支持强；可只下载字幕；覆盖 AI 字幕路径	外部二进制依赖；集成和调试成本更高；并不适合作为第一步变更

表 1: 围绕现有 pipeline 的维护者视角比较

5 建议的目标架构

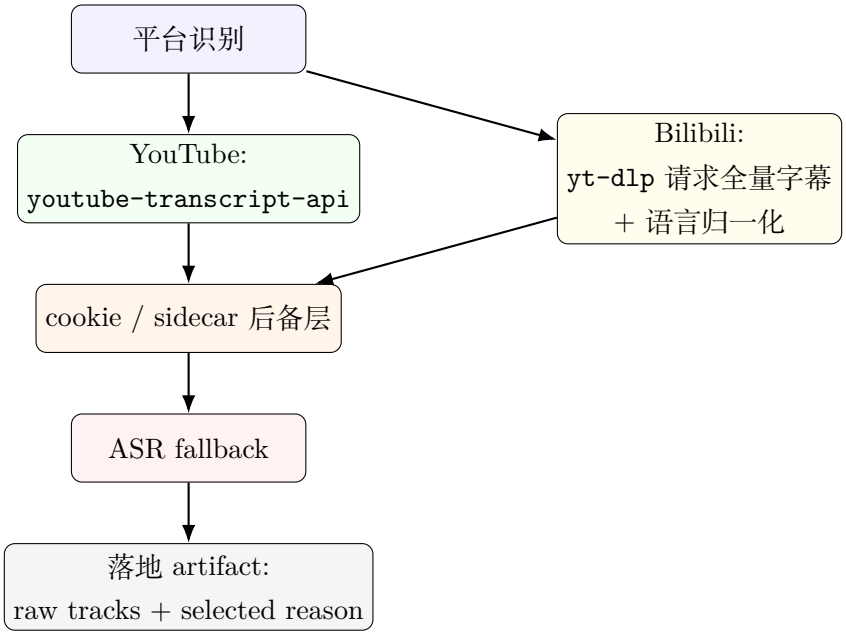

图 1: 建议的字幕获取分层

5.1 YouTube：维持现状的主方向，不建议大改

建议顺序：

- 第一层：`youtube-transcript-api.list()` 获取公开 transcript 列表，并优先人工字幕
- 第二层：`yt-dlp` cookie/sidecar 路径处理年龄限制、需要登录或 sidecar-only 的情况
- 第三层：ASR fallback

这个路径的优点是各层职责清晰，且已被现有工程验证。

5.2 Bilibili：先修策略，再决定是否引入专用层

建议的近期改动是：

- 不再把 `subtitleslangs` 限制在固定少量语言码；至少要支持 `all,-live_chat,-danmaku`
- 为 B 站增加语言归一化，把 `ai-zh`、`zh-Hans`、`zh-CN`、`zh-Hant`、`zh-TW`、`yue`、`auto-zh` 收敛到统一的中文候选族
- 先做“归一化之后的偏好排序”，再决定人工/自动字幕优先级
- 在 `subtitle_probe.json` 中保留原始轨道清单和未命中原因，而不是只返回 `none`

最有性价比的 change

对 B 站来说，最值得立刻实现的不是“换库”，而是“请求全量字幕 + 平台特定语言归一化 + artifact 可观测性”。这三个改动能直接修掉当前样例问题，并且对后续真实 case 调试最有帮助。

5.3 什么时候再考虑 B 站专用库

只有在下面任一情况持续出现时，才建议上升到第二阶段：

- 即使请求全量字幕并做语言归一化，`yt-dlp` 仍显著漏掉 B 站官方字幕
- 需要直接使用 `cid`、字幕列表、翻译轨、分 P 语义等更细粒度能力
- 需要更强的登录态、反风控或 TV / App 鉴权控制

这时可以考虑两条路：

- 轻量路线：自己实现一个极小的 B 站 direct API adapter，只做 `cid` 与字幕列表查询
- 重量路线：接入 `bilibili-api-python` 或 `BBDown` 作为 B 站专用 fallback

6 为后续 propose 准备的变更建议

如果下一步要开 OpenSpec change，我建议目标写成“修复 B 站字幕探测的语言覆盖与可观测性”，而不是“替换字幕引擎”。可以拆成四项：

1. 将 B 站字幕请求从固定语言集合改为全量请求或正则请求，并显式排除 `live_chat` / `danmaku`。
2. 增加 B 站语言归一化与优先级选择，覆盖 `ai-zh` 这类自动字幕语言码。
3. 扩展 `subtitle_probe.json`，保留 raw tracks、归一化结果与未命中原因。
4. 增加真实回归测试，至少覆盖“cookies 有效且字幕语言码为 `ai-zh`”这一类 case。

暂缓事项

不建议把“接入 `BBDown`”或“全面迁移到 `bilibili-api-python`”放进第一轮 change。它们可以在 P1/P2 里继续调研，但不该和当前这个已知、局部、可复现的问题绑在一起。

7 结论与后续问题

7.1 已经有强证据支持的判断

- 当前 B 站字幕 miss 的直接原因是 adapter 自己的语言覆盖与选择逻辑过窄。
- `yt-dlp` 具备获取 B 站该样例官方字幕的能力，当前不需要因为这个问题整体替换它。
- `youtube-transcript-api` 仍然适合作为 YouTube 第一层 transcript 接口。

7.2 很可能成立，但还值得继续验证的判断

- B 站还会继续出现更多非 `zh-Hans` 命名的字幕标签，因此语言归一化应该做成平台策略，而不是一次性补 `ai-zh`。
- 即使未来引入 B 站专用 API 层，`yt-dlp` 仍然值得保留为跨平台 fallback。

7.3 还不知道，但下一轮 change 应该顺手补上的观测点

- 请求全量字幕后，不同 B 站视频会返回哪些语言标签分布
- 哪些失败是真正“平台没有字幕”，哪些失败只是“轨道没被挑中”
- 在修复语言归一化后，是否还有足够多的 case 值得引入 B 站 direct API adapter

A 来源清单

Source ID	模态	标题 / 标签	链接或路径
S1	Web	`yt-dlp` 官方 README	https://github.com/yt-dlp/yt-dlp
S2	Web	`youtube-transcript-api` 官方 README	https://github.com/jdepoix/youtube-transcript-api
S3	Web	`BBDown` 官方 README	https://github.com/nilaoda/BBDown
S4	Web	`bilibili-api-python` 官方 README	https://github.com/Nemo2011/bilibili-api
S5	Web	B 站 AI 字幕接口公开 issue 讨论	https://github.com/SocialSisterYi/bilibili-API-collect/issues/709
L1	Code	当前 B 站 adapter 实现	`/home/fanghaotian/Desktop/src/video-note-pipeline/src/video_note_pipeline/adapters/base.py`
L2	JSON	当前样例 B 站 probe artifact	`/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-subtitle-strategy/artifacts/current-subtitle-probe-BV1DzALzQEHU.json`
L3	SRT	直接调用 `yt-dlp` 获得的 `ai-zh` 字幕	`/home/fanghaotian/Desktop/src/agent-basic-skill/reports/2026-03-26-bilibili-subtitle-strategy/artifacts/direct-yt-dlp-BV1DzALzQEHU.ai-zh.srt`